

Design Patterns

Advanced Programming
Brent van Bladel



Design Principles

- Liskov substitution principle
- Type Conformity
- Principle of Closed Behavior
- Single-responsibility principle
- Open–closed principle
- Low coupling / high cohesion

Object-oriented designs following these principles are more understandable, flexible, and maintainable.

Single-responsibility principle

“Every class must have only one responsibility.”

```
1. class FileManager {  
2.     public:  
3.         void write(string filename, string text);  
4.         string read(string filename);  
5.     };
```

Violates SRP

```
1. class FileWriter {  
2.     public:  
3.         void write(string filename, string text);  
4.     };  
5.  
6. class FileReader {  
7.     public:  
8.         string read(string filename);  
9.     };  
10.  
11. class FileManager  
12.     : public FileWriter, public FileReader {};
```

Follows SRP

Open–closed principle

*“Software entities should be open for extension,
but closed for modification”*

```
1. class FileReader {  
2.     XMLReader _xmlReader;  
3.     JSONReader _jsonReader;  
4. public:  
5.     string read(string filename) {  
6.         if (endsWith(filename, "XML")  
7.             return _xmlReader.readXML(filename);  
8.         else if (endsWith(filename, "JSON")  
9.             return _jsonReader.readJSON(filename);  
10.    };  
11. };
```

Violates OCP

```
1. class FileReader {  
2. public:  
3.     virtual string read(string filename);  
4. };  
5. class XMLReader : public FileReader {  
6. public:  
7.     string read(string filename) override;  
8. };  
9. class JSONReader : public FileReader {  
10. public:  
11.     string read(string filename) override;  
12. };
```

Follows OCP

Low coupling / high cohesion

Coupling is the degree to which the different classes depend on each other.

→ Should be as low as possible.

Cohesion is the degree to which the elements of a class belong together.

→ Should be as high as possible.

Design Pattern

A **design pattern** provides a typical solution to a common problem in software design.

- Tried and tested solutions to common problems.
- Follow design principles.
- Define a common language that developers can use to communicate more efficiently.

Design Pattern

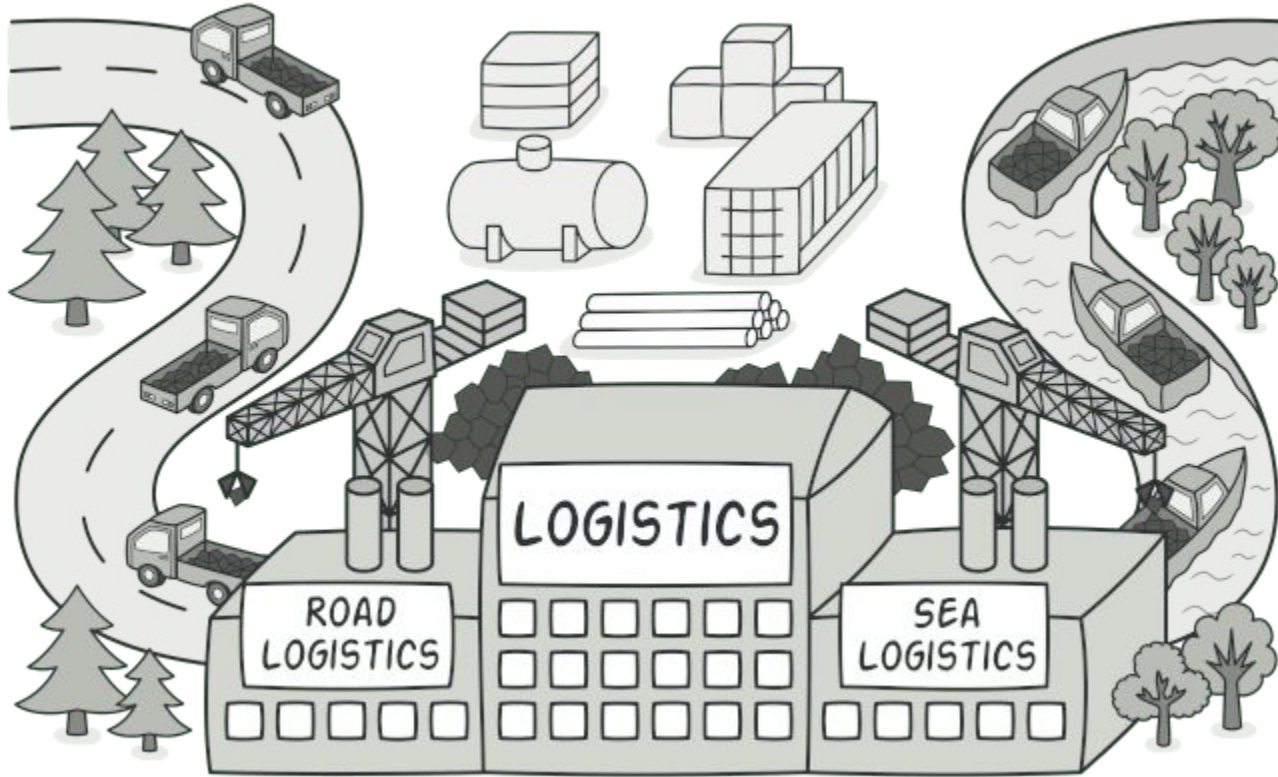
For each pattern, we will examine:

- the **problem** the pattern tries to solve.
- the **solution** that solves the problem.
- the **structure** which shows the relation between the different parts of the pattern.
- the **benefits** and the **drawbacks** of the pattern.

Creational Patterns

These patterns provide various object creation mechanisms, which increase flexibility and reuse of existing code.

Factory Method



Factory Method: Problem

```
1.  if (request == "Circle") {  
2.      Circle c();  
3.      cout << c.area();  
4.  } else if (request == "Rectangle") {  
5.      Rectangle r();  
6.      cout << r.area();  
7.  }
```

- Given some variable, we want to construct a certain object.
→ We end up with many conditionals in our code that switch behavior depending on the class.
- What if we need to add a new class?

Factory Method: Solution

Factory Method

```
1. Shape* shapeFactory(string request) {  
2.     if (request == "Circle")  
3.         return new Circle();  
4.     if (request == "Rectangle")  
5.         return new Rectangle();  
6.     return nullptr;  
7. }
```

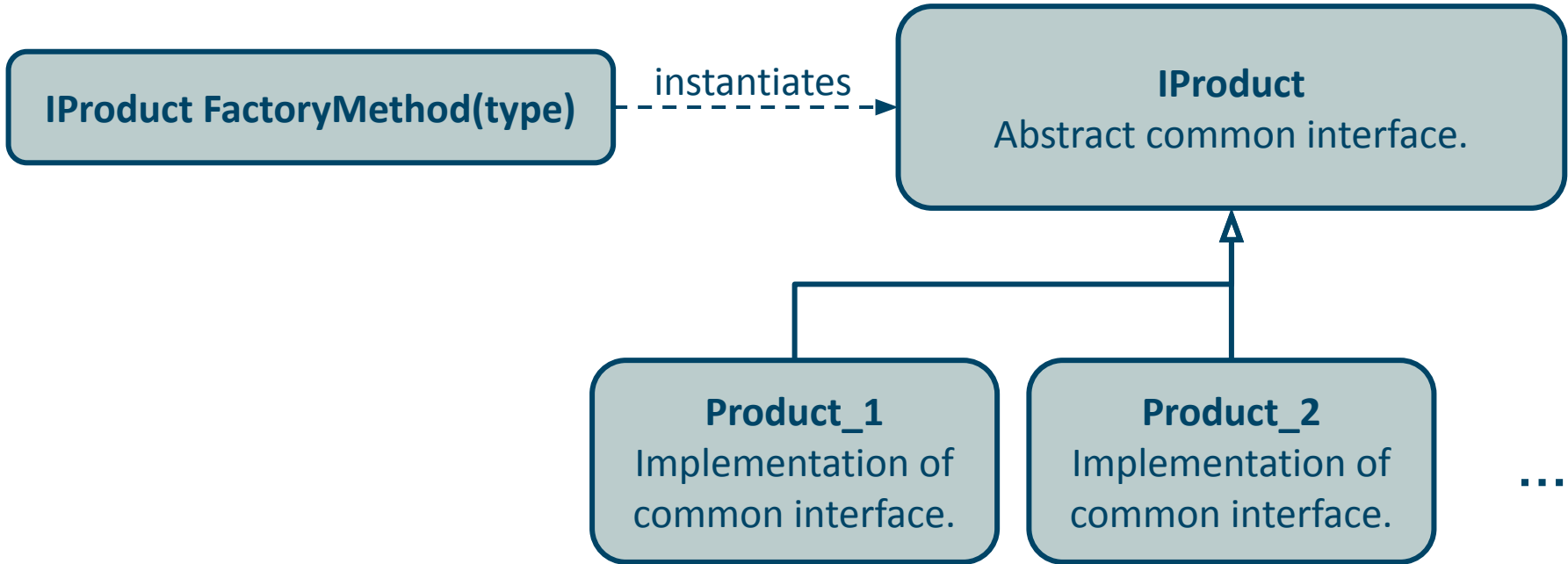
```
1. ...  
2. Shape* s = shapeFactory(request);  
3. cout << s->area();  
4. ...
```

Common interface

```
1. class Shape{  
2.     public:  
3.         virtual float area() const = 0;  
4. };  
5. class Circle: public Shape{  
6.     public:  
7.         float area() const override;  
8. };  
9. class Rectangle: public Shape{  
10.    public:  
11.        float area() const override;  
12.    };
```

Replace direct object construction calls with calls to a special factory method.

Factory Method: Structure



Factory Method: Benefits & Drawbacks

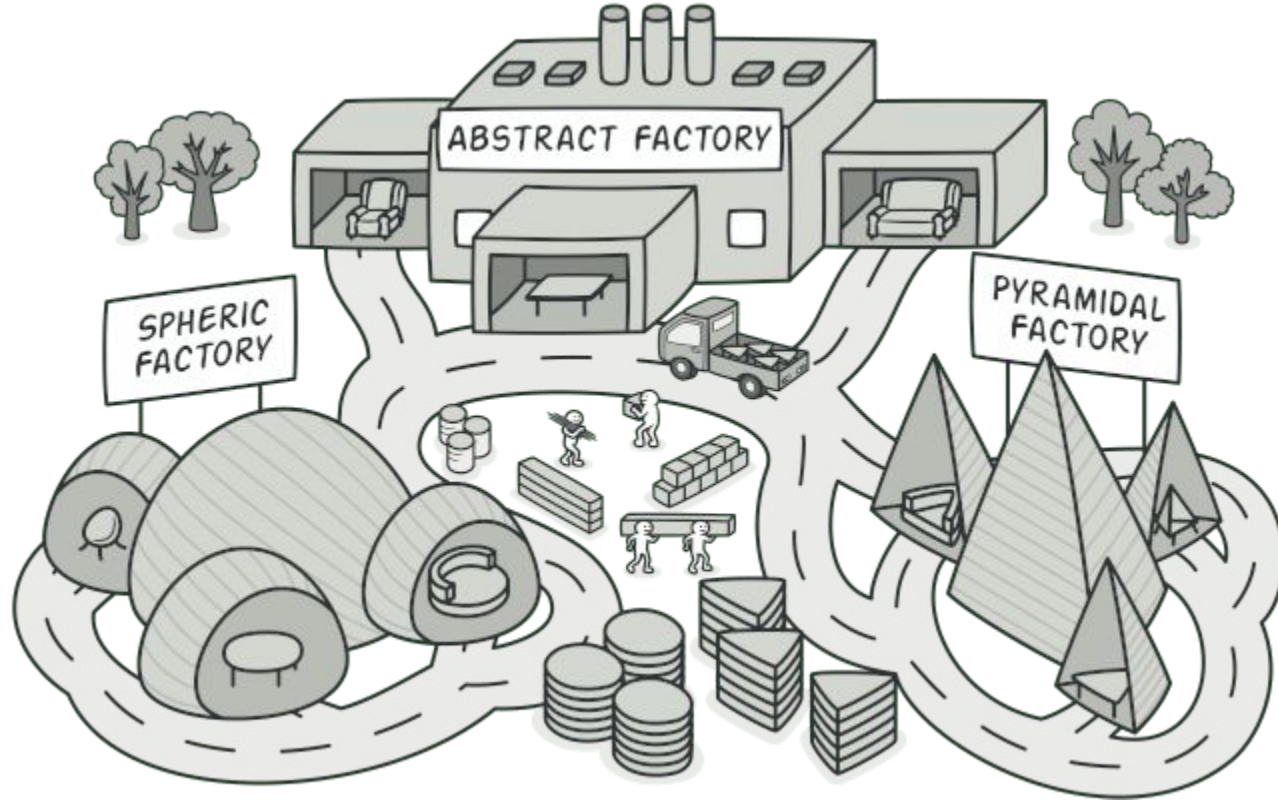
Benefits

- Avoid close coupling between the creator and the concrete products.
- **Single Responsibility Principle.** You can move the product creation code into one place in the program, making the code easier to support.
- **Open/Closed Principle.** You can introduce new types of products into the program without breaking existing client code.

Drawbacks

- The code may become more complicated since you need to introduce a lot of new subclasses.

Abstract Factory



Abstract Factory: Problem

```
1. Shape* shapeFactory(string shape, string type) {  
2.     if (request == "Rectangle") {  
3.         if (type == "size")  
4.             return SizedRectangle();  
5.         if (type == "coordinates")  
6.             return CoordinatedRectangle();  
7.     } else if (...) {  
8.         ...
```

```
1.     class SizedRectangle: public Rectangle {  
2.         float _width, _length;  
3.     public:  
4.         float area() const override;  
5.     };  
6.     class CoordinatedRectangle:  
7.         public Rectangle {  
8.         vector<Coordinate> _corners;  
9.     public:  
10.        float area() const override;  
11.    };
```

- We want to construct certain objects via a factory method, but each object has multiple implementations.
- How can we guarantee that the constructed objects work together?

Abstract Factory: Solution

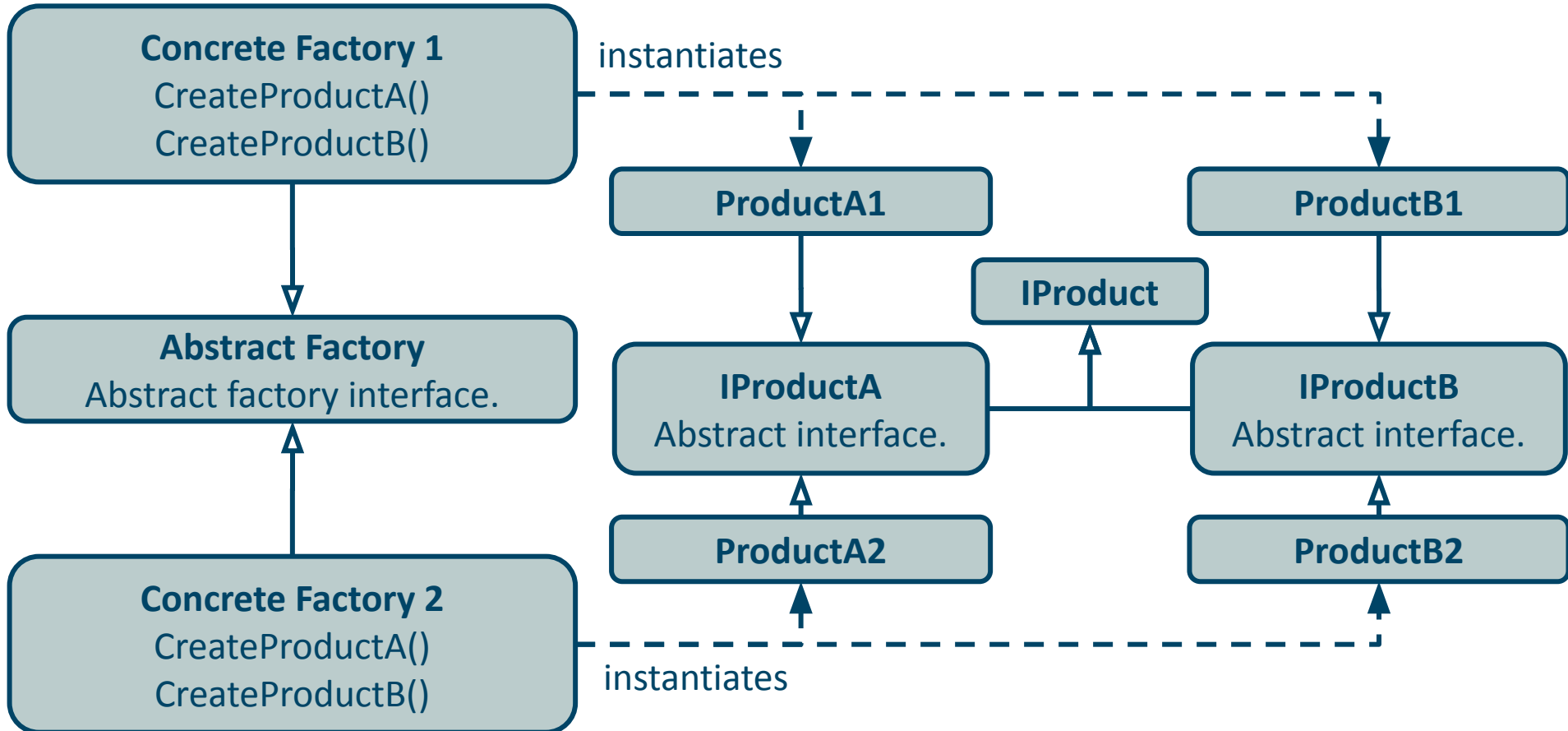
```
1. ShapeFactory* f = SizedShapeFactory();  
2.  
3. Circle* c = f->createCircle();  
4. Rectangle* r = f->createRectangle();
```

Place the factory methods in a class that implement an abstract factory interface.

Abstract Factory

```
1. class ShapeFactory {  
2.     public:  
3.         virtual Shape* createRectangle() = 0;  
4.         virtual Shape* createCircle() = 0;  
5.     };  
6. class SizedShapeFactory: public ShapeFactory {  
7.     public:  
8.         Shape* createRectangle() override;  
9.         Shape* createCircle() override;  
10.    };  
11. class CoordinatedShapeFactory:  
12.     public ShapeFactory {  
13.     public:  
14.         Shape* createRectangle() override;  
15.         Shape* createCircle() override;  
16.    };
```


Abstract Factory: Structure



Abstract Factory: Benefits & Drawbacks

Benefits

- Guarantee that the created products from a factory are compatible.
- Avoid close coupling between the creator and the concrete products.
- **Single Responsibility Principle.** You can move the product creation code into one place in the program, making the code easier to support.
- **Open/Closed Principle.** You can introduce new types of products into the program without breaking existing client code.

Drawbacks

- The code may become more complicated since a lot of new interfaces and classes are introduced.

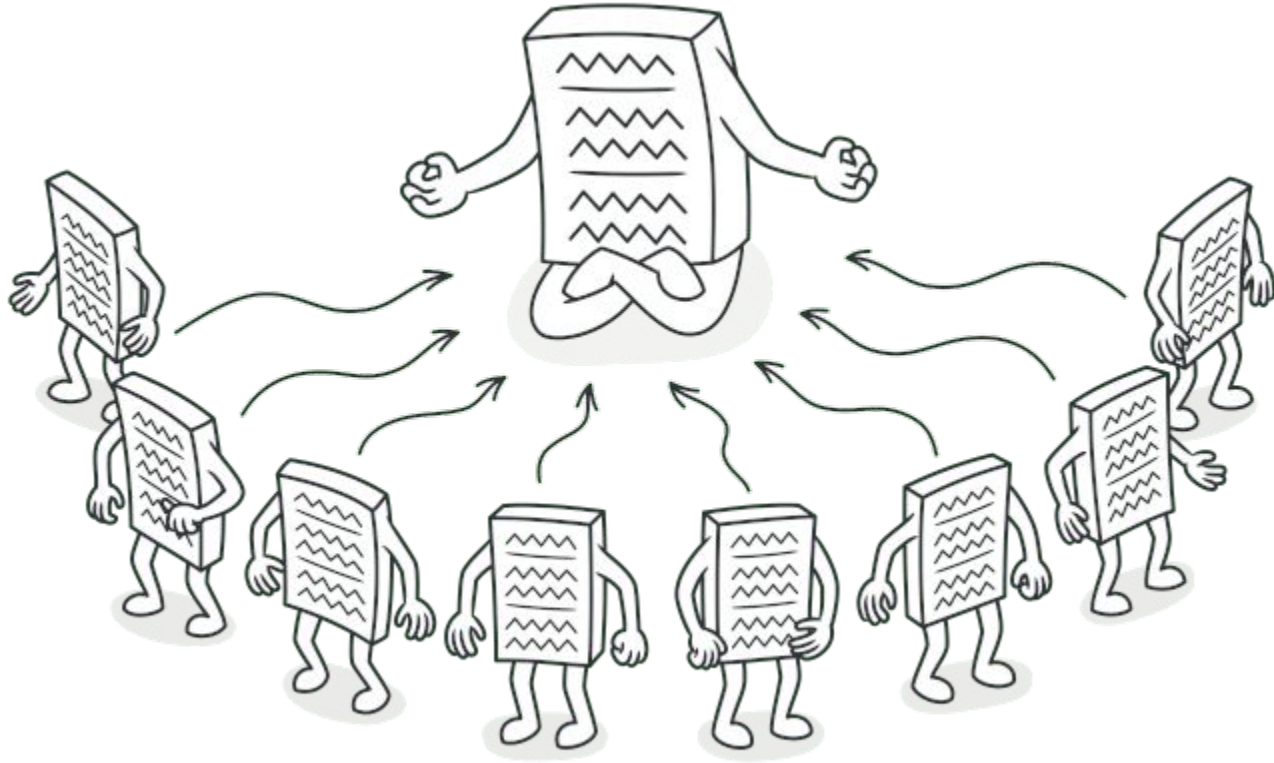
Combining Design Patterns

```
1. class ShapeFactory {
2.     public:
3.         virtual Shape* createShape(string request) = 0;
4. };
5.
6. class SizedShapeFactory: public ShapeFactory {
7.     public:
8.         Shape* createShape(string request) override;
9. };
10.
11. class CoordinatedShapeFactory: public ShapeFactory {
12.     public:
13.         Shape* createShape(string request) override;
14. };
```

Abstract Factory

Factory Method

Singleton



Singleton: Problem

```
1.  ...  
2.  ShapeFactory* f1 = SizedShapeFactory();  
3.  Shape* s1 = shapeFactory->createRectangle();  
4.  ...  
5.  ShapeFactory* f2 = CoordinatedShapeFactory();  
6.  Shape* s2 = shapeFactory->createCircle();  
7.  ...
```

- We want to have exactly one object of a certain class in our program.
- How can we guarantee that only one object is created?

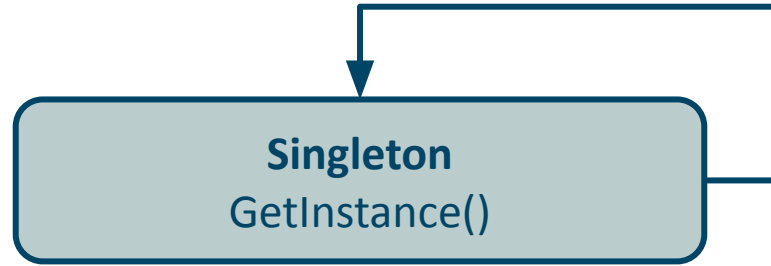
Singleton: Solution

```
1. class ShapeFactory {  
2.     private:  
3.         static ShapeFactory* _instance;  
4.         ShapeFactory() = default;  
5.     public:  
6.         ~ShapeFactory() = default;  
7.  
8.         static ShapeFactory* getInstance() {  
9.             if (!_instance)  
10.                 _instance = new ShapeFactory();  
11.             return _instance;  
12.         }  
13.  
14.         Shape* createShape(string request);  
15.     };
```

1) Make the constructor private. Have a private static member containing the singleton.

2) Create a static method that creates the singleton the first time it is called. All following calls to this method return the already created object.

Singleton: Structure



Singleton: Benefits & Drawbacks

Benefits

- Guarantee that a class has only one instance.
- Global access point to the instance.
- The singleton object is initialized only when it's requested for the first time.

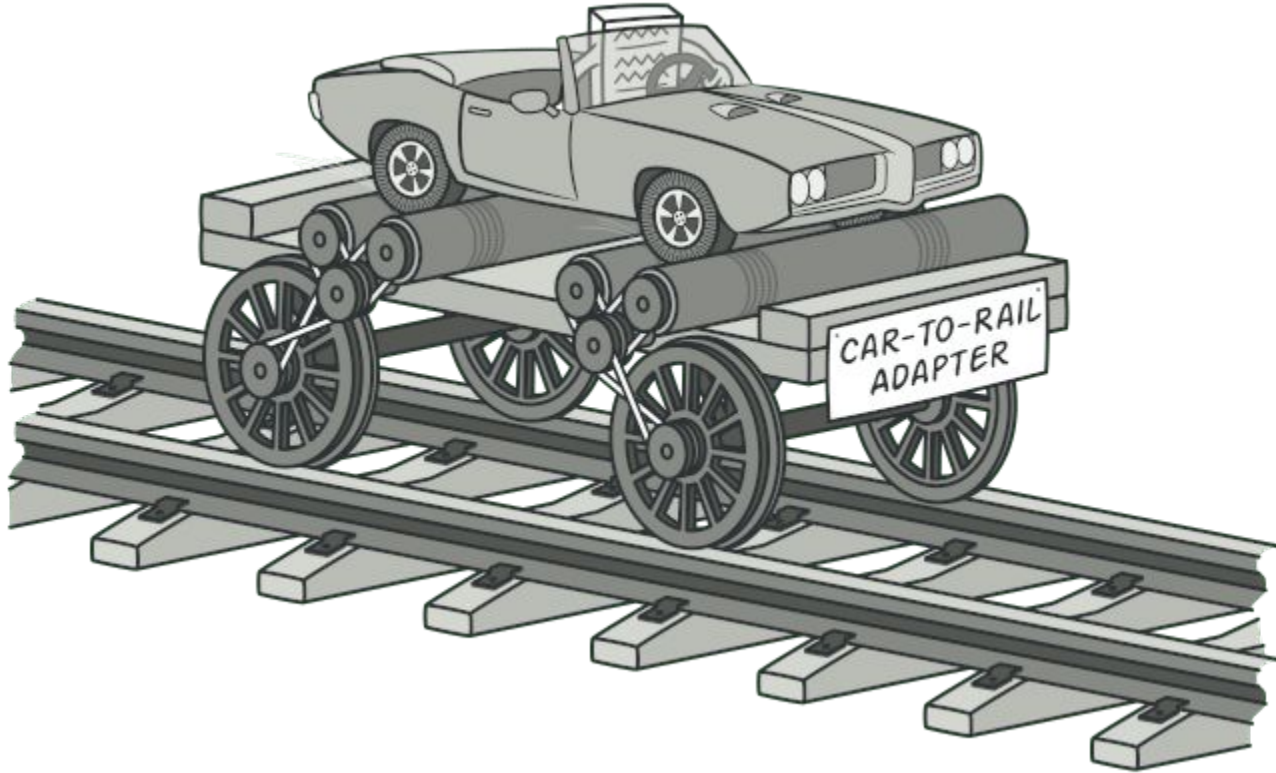
Drawbacks

- Violates the **Single Responsibility Principle**.
- Can cause a bad design. They hide the dependencies of your application, and they inherently cause code to be tightly coupled.
- Requires special treatment in a multithreaded environment.

Structural Patterns

These patterns explain how to assemble objects and classes into larger structures, while keeping these structures flexible and efficient.

Adapter



Adapter: Problem

```
1. ...  
2. MyCircle c;  
3. LibSquare s;  
4.  
5. cout << c.area() << endl;  
6. cout << s.calculateArea()  
   << endl;  
7. ...
```

```
1. class MyCircle : public MyShape {  
2.     public:  
3.         float area() override;  
4. };  
5.  
6. class LibSquare : public LibShape{  
7.     public:  
8.         float calculateArea() override;  
9. };
```

- We want to use two similar classes with a different interface.
- How can we call these interchangeably?

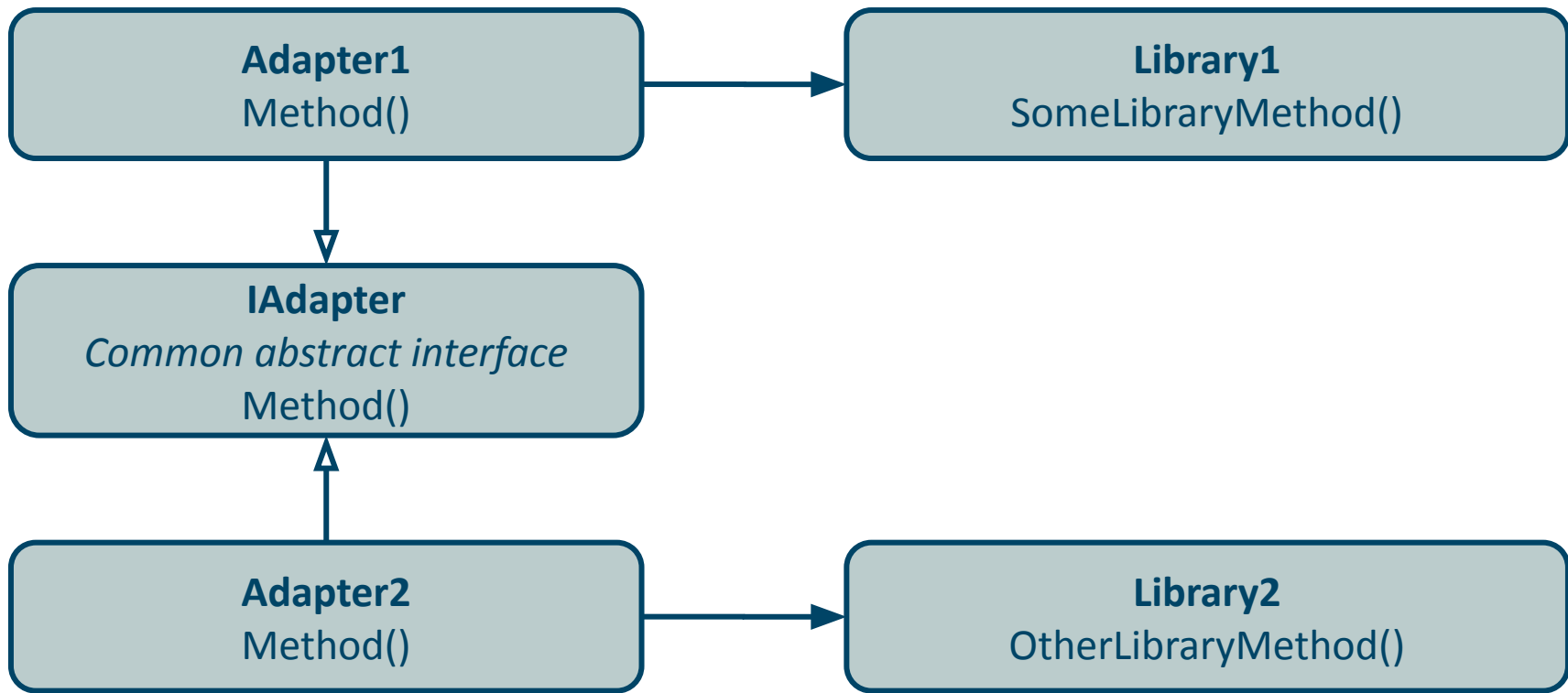
Adapter: Solution

```
1. ...
2. vector<MyShape *> shapes;
3.
4. shapes.push_back(new MyCircle());
5. shapes.push_back(new MyRectangleAdapter());
6.
7. for (auto shape: shapes)
8.     cout << shape->area() << endl;
9. ...
```

```
1. class MySquareAdapter : public MyShape {
2.     private:
3.         LibSquare _square;
4.     public:
5.         float area() override {
6.             return _square.calculateArea();
7.         };
8.     };
```

Implement an adapter in one interface that wraps the other class

Adapter: Structure



Adapter: Benefits & Drawbacks

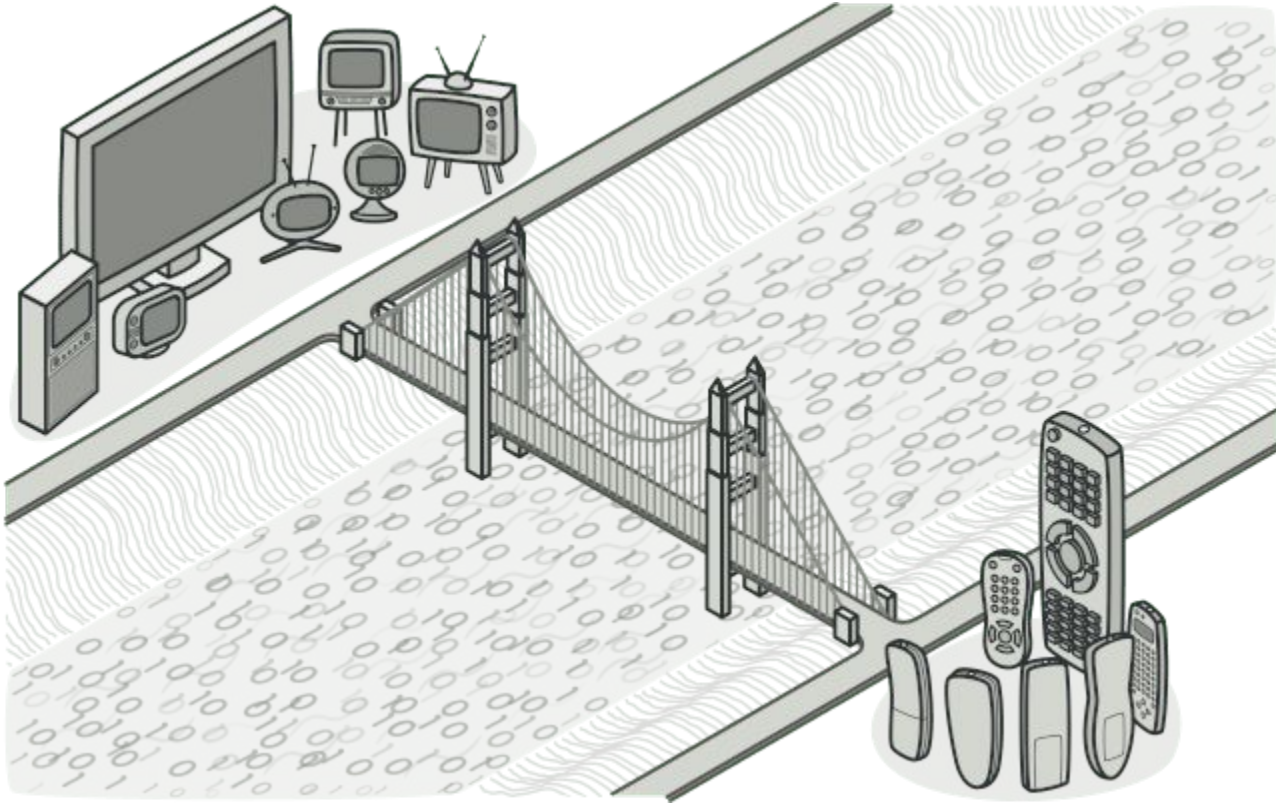
Benefits

- **Single Responsibility Principle.** Separate the interface or data conversion code from the primary business logic of the program.
- **Open/Closed Principle.** Introduces new types of adapters into the program without breaking the existing client code.

Drawbacks

- The overall complexity of the code increases because you need to introduce a set of new interfaces and classes.

Bridge



Bridge: Problem

```
1. class Shape {
2.     public:
3.         virtual float area() = 0;
4.         virtual void printColor() = 0;
5. };
6.
7. class RedCircle : public Shape {
8.     public:
9.         float area() override;
10.        void printColor() override;
11. }
12. class BlueCircle : public Shape {
13.     public:
14.         float area() override;
15.         void printColor() override;
16. }
```

```
1. class RedSquare : public Shape {
2.     public:
3.         float area() override;
4.         void printColor() override;
5. }
6.
7. class BlueSquare : public Shape {
8.     public:
9.         float area() override;
10.        void printColor() override;
11. }
```

- We want to specialize a class.
Example: platform dependent classes.
- Adding a new class or specialization makes the hierarchy grow exponentially.

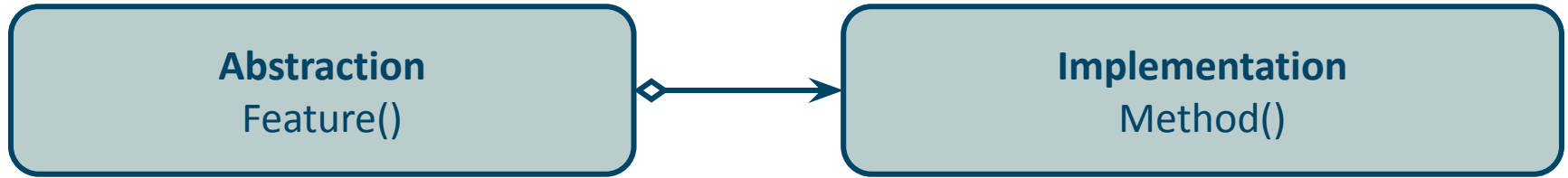
Bridge: Solution

```
1. class Shape {  
2.     private:  
3.         Color _color;  
4.     public:  
5.         virtual float area() = 0;  
6.         virtual void printColor() = 0;  
7. };
```

Switch from inheritance to object composition.

→ Extract one of the dimensions into a separate class hierarchy.

Bridge: Structure



Bridge: Benefits & Drawbacks

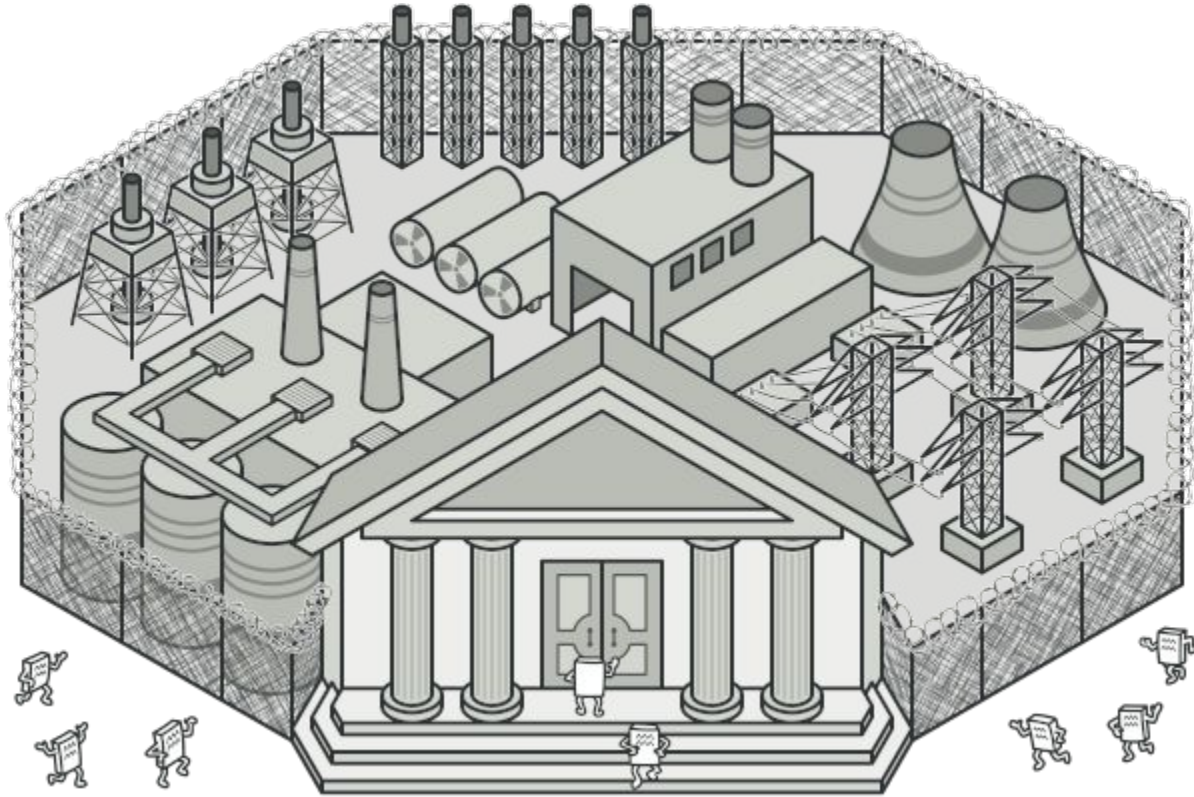
Benefits

- The caller works with high-level abstractions. It isn't exposed to the details.
- **Single Responsibility Principle.** Focus on high-level logic in the abstraction and on low-level details in the implementation.
- **Open/Closed Principle.** Introduce new abstractions and implementations independently from each other.

Drawbacks

- The overall complexity of the code can increase when applying the bridge pattern to a highly cohesive class.

Facade



Facade: Problem

```
1.  class FuelTank{
2.      public:
3.          Fuel* getFuel();
4.  };
5.  class Engine {
6.      public:
7.          void start();
8.          void consumeFuel();
9.          void chargeBattery();
10.         void drive();
11.         void stop();
12.  };
13.  class HeadLights {
14.      public:
15.          void consumeBattery();
16.  };
```

- We want to use a bunch of classes that form a complex subsystem.
- Using this subsystem requires us to keep many objects and call specific functions in the correct order.

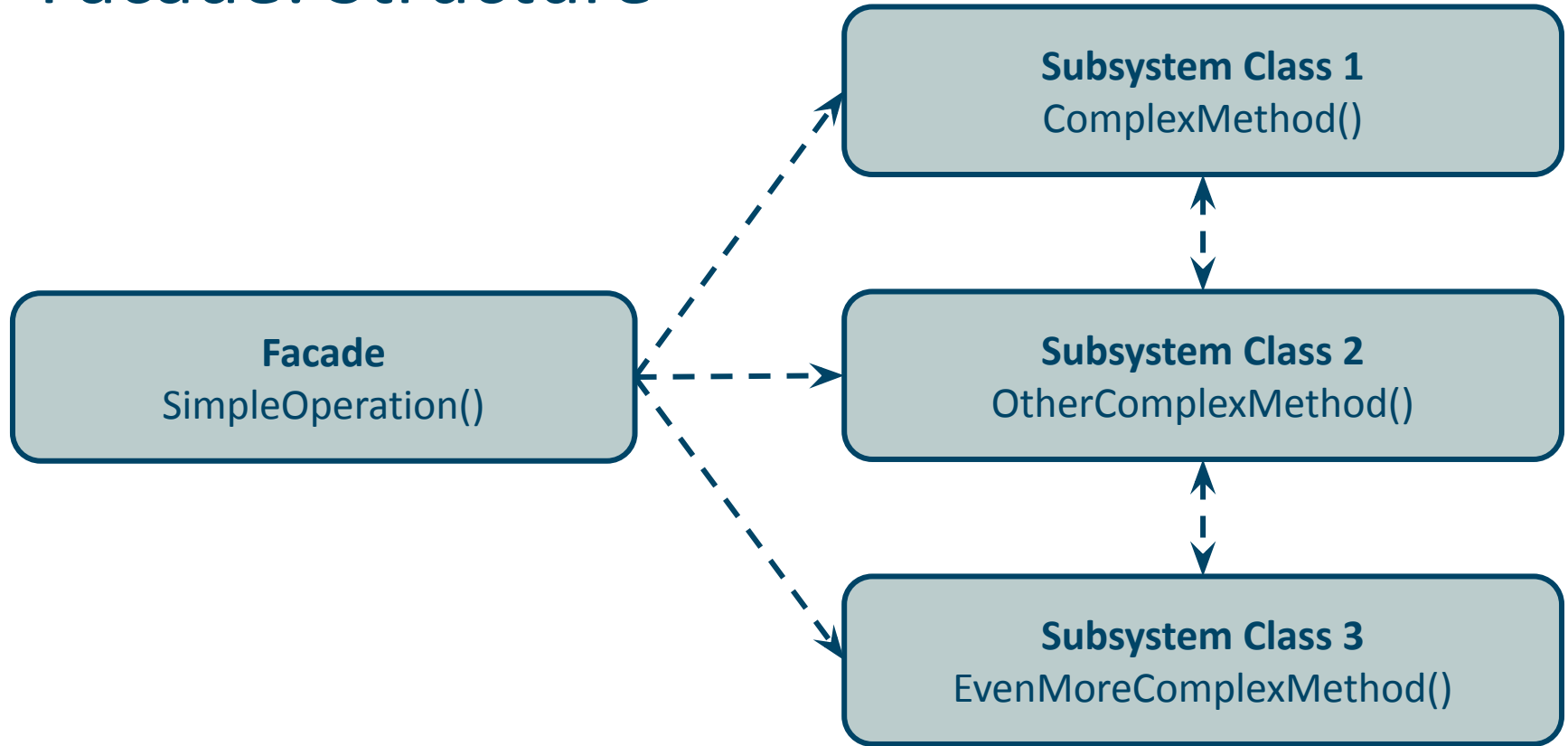
Facade: Solution

```
1. class Car {  
2.     private:  
3.         FuelTank _tank;  
4.         Engine _engine;  
5.         HeadLights _lights;  
6.     public:  
7.         void drive();  
8. };
```

Implement a facade class that manages the objects and handles all the complex calls.

→ The user only needs to create the facade and call the simpler facade calls.

Facade: Structure



Facade: Benefits & Drawbacks

Benefits

- Isolate your code from the complexity of a subsystem.

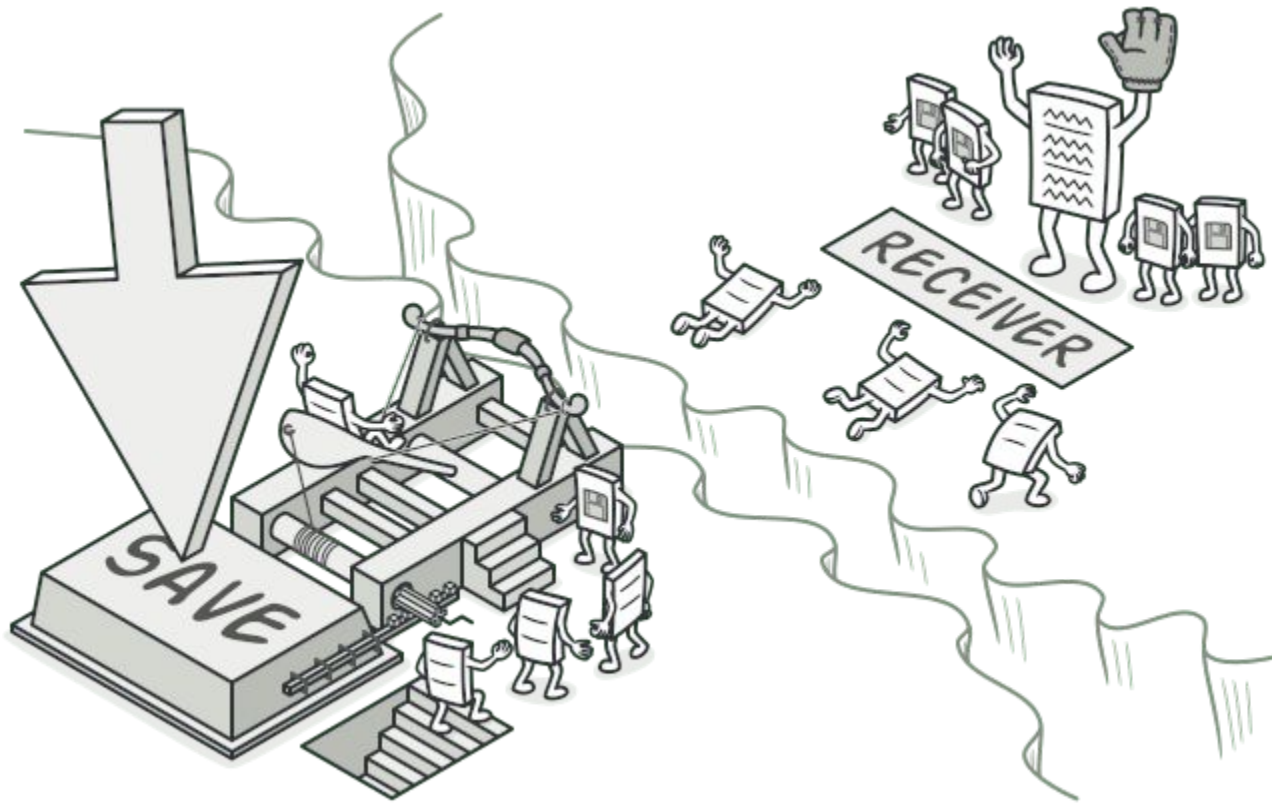
Drawbacks

- A facade can become a god object coupled to all classes of the subsystem.

Behavioral patterns

These patterns are concerned with algorithms and the assignment of responsibilities between objects.

Command



Command: Problem

```
1. class LightOnButton : public Button {  
2.     public:  
3.         void press() override;  
4. };  
5. class LightOffButton : public Button {  
6.     public:  
7.         void press() override;  
8. };  
9. class SoundOnButton : public Button {  
10.    public:  
11.        void press() override;  
12. };  
13. class SoundOffButton : public Button {  
14.    public:  
15.        void press() override;  
16. };
```

- We have a bunch of classes that each perform the same action on different targets.
- Adding a new target will require us to create many additional classes.

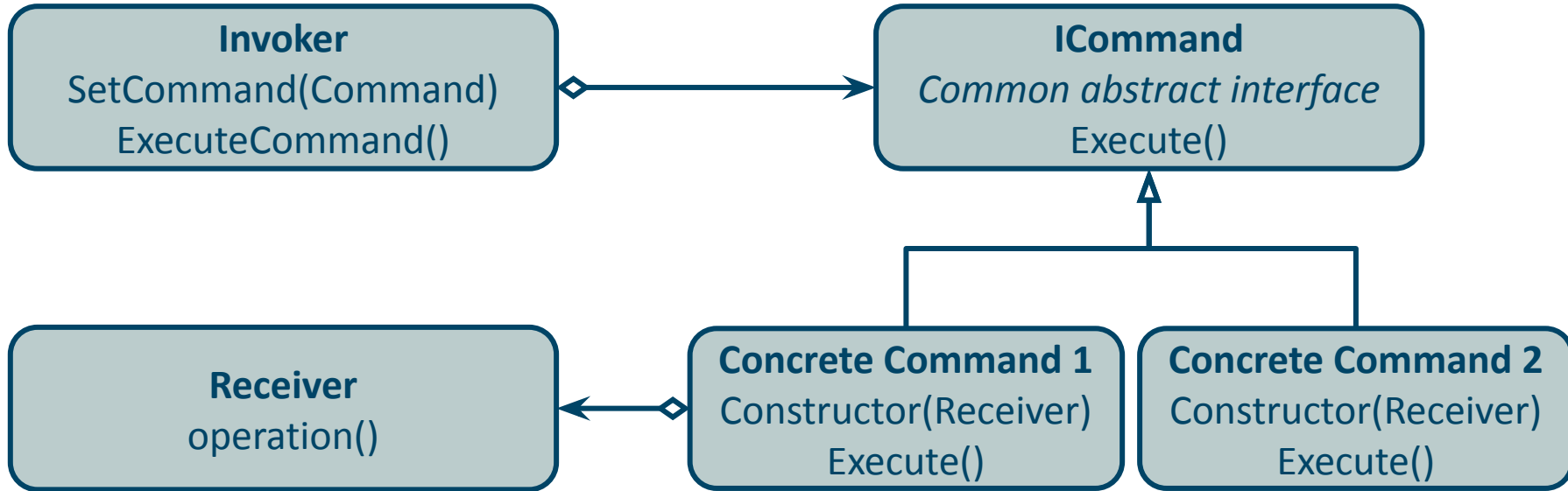
Command: Solution

Implement a class that represents the action.

→ Adding a new target does not increase the amount of classes.

```
1. class Command {
2.     Device* _device;
3. };
4.
5. class TurnOnCommand: public Command {
6. public:
7.     TurnOnCommand(Device* d) : _device{d} { };
8.     void execute() override { _device->on(); };
9. };
10. class TurnOffCommand: public Command {
11. public:
12.     TurnOnCommand(Device* d) : _device{d} { };
13.     void execute() override { _device->off(); };
14. };
15.
16. class Button {
17. public:
18.     Command* press();
19. };
```

Command: Structure



Command: Benefits & Drawbacks

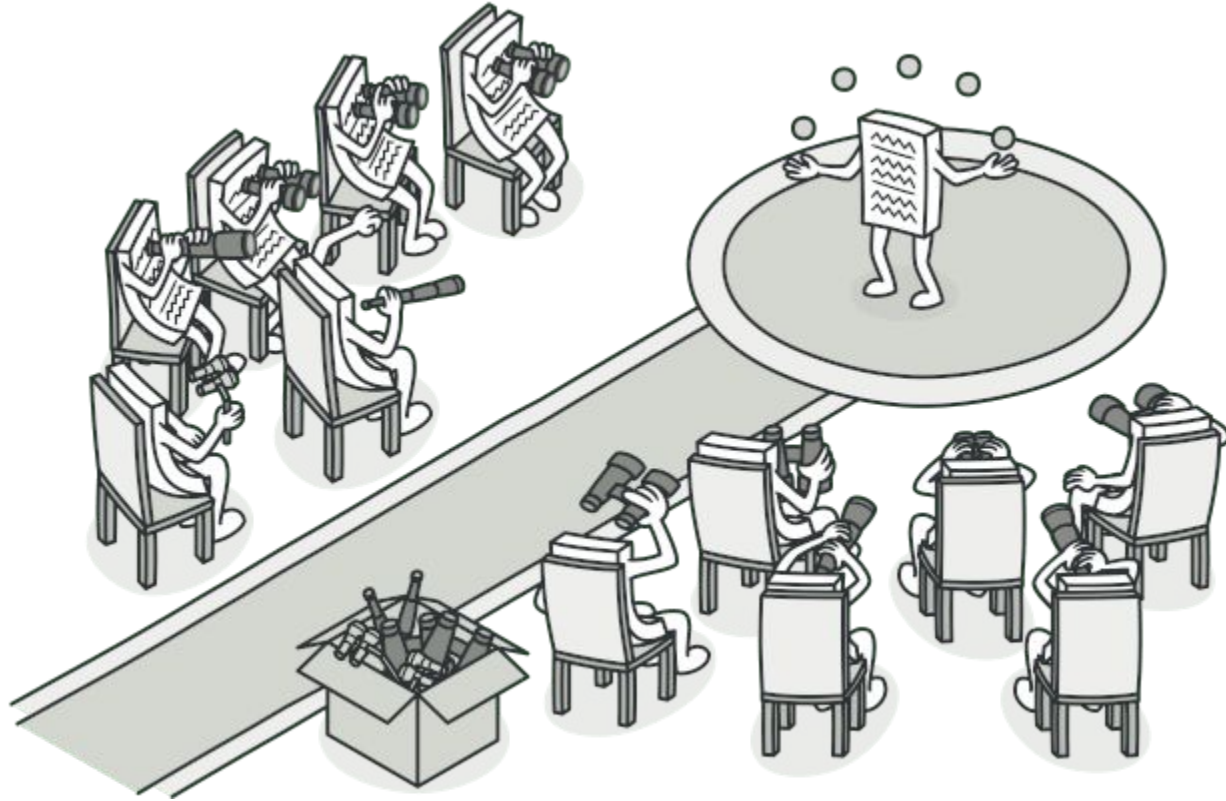
Benefits

- **Single Responsibility Principle.** Decouple the classes that invoke operations from the classes that perform these operations.
- **Open/Closed Principle.** Introduce new commands without breaking existing client code.
- Simple commands can be combined into a complex one.

Drawbacks

- The code may become more complicated since you're introducing a whole new layer between senders and receivers.

Observer



Observer: Problem

```
1. class Weather {
2.     float _temperature;
3.     bool _isRaining;
4.     public:
5.         void setWeatherData(float t, bool r);
6. };
7.
8. class TemperatureDisplay {
9.     public:
10.        void display(float t);
11. };
12. class WaterCollector {
13.     public:
14.         void turnOn();
15.         void turnOff();
16. };
```

- We have classes that are interested in the state of an object and need to be updated when that state changes.
- What if we need to add a new class?

Observer: Solution

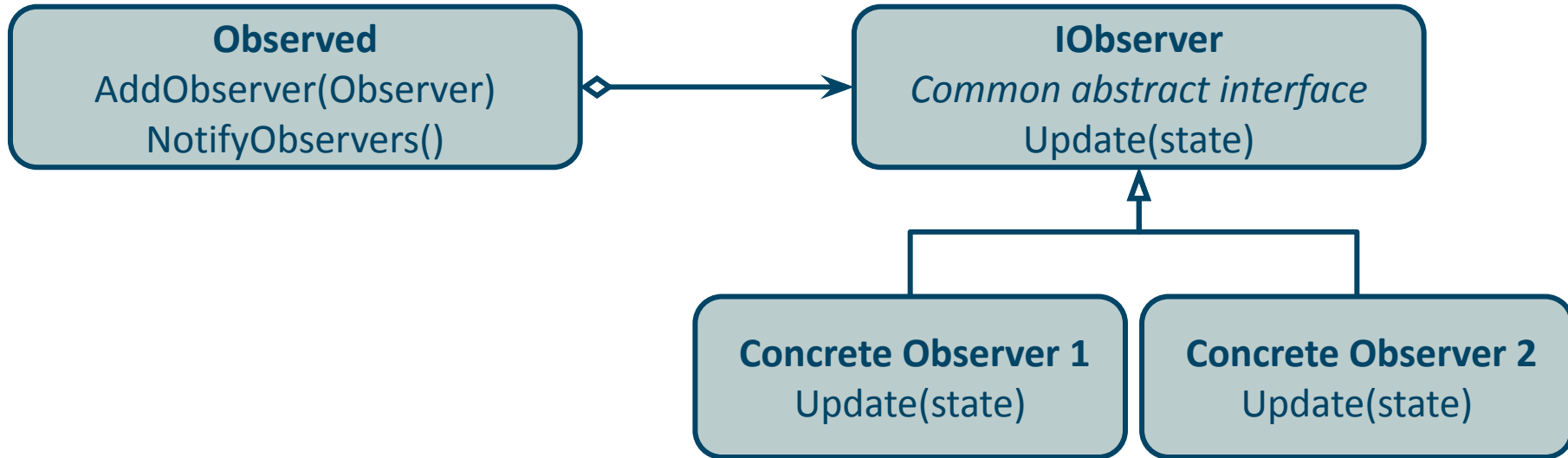
```
1. class Weather {
2.     float _temperature;
3.     bool _isRaining;
4.     list<Observer*> _observers;
5. public:
6.     void addObserver(Observer* o);
7.
8.     void setWeatherData(float t, bool r){
9.         _temperature = t;
10.        _isRaining = r;
11.        for (auto observer: _observers)
12.            observer.update(this);
13.    }
14.};
```

```
1. class Observer {
2.     public:
3.         virtual void update(Weather* w) = 0;
4. };
5. class TemperatureDisplay : public Observer {
6.     public:
7.         void update(Weather* w) {
8.             display(w->getTemp()); };
9.         void display(float t);
10.    };
```

Add a common interface to all interested classes, and store them in a list.

→ update all list items on state change

Observer: Structure



Observer: Benefits & Drawbacks

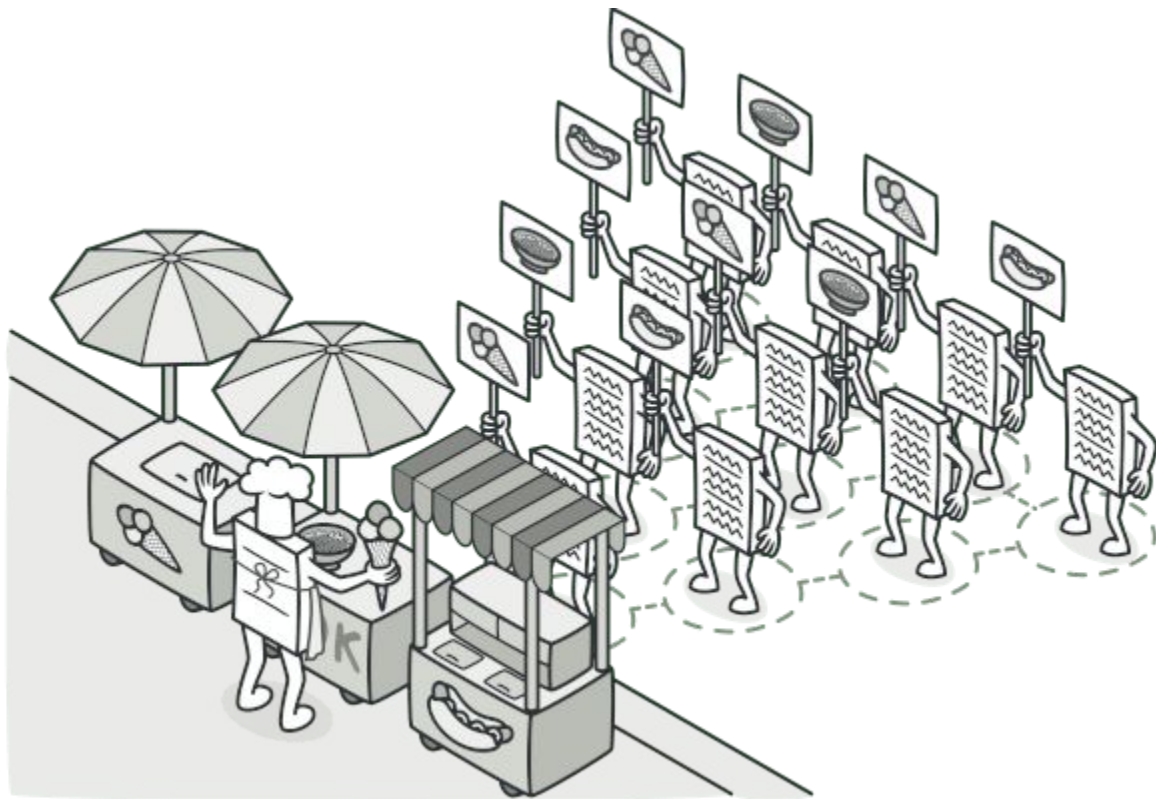
Benefits

- **Open/Closed Principle.** Introduce new observer classes without changing the observed class.
- Establish relations between objects at runtime.

Drawbacks

- Subscribers are notified in random order.
- Violates **Single Responsibility Principle** as the observed class needs to both maintain a list of observers and its original functionality.

Visitor



Visitor: Problem

```
1. class AbstractNode{
2.     int _data;
3. };
4.
5. class RootNode {
6.     vector<AbstractNode*> _children;
7. };
8.
9. class Node : public AbstractNode {
10.     vector<AbstractNode*> _children;
11. };
12.
13. class FinalNode : public AbstractNode {
14.     };
```

- We have a complex datastructure consisting of multiple interconnected classes.
- We want to add functionality that goes through the entire datastructure.

Example: print function,
export to XML function,
...

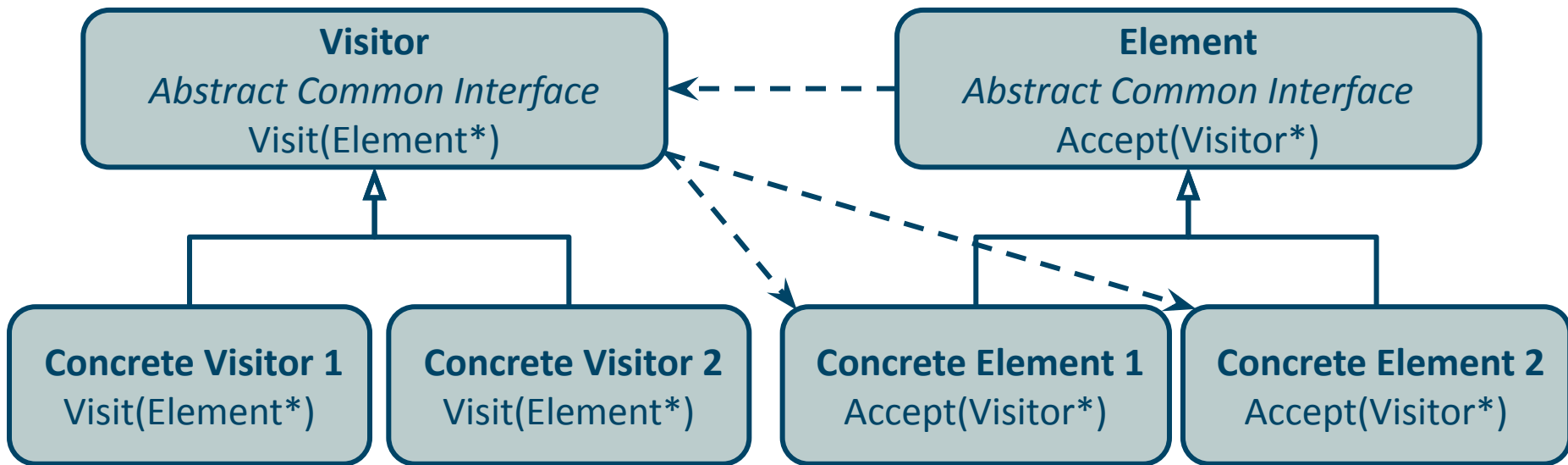
Visitor: Solution

```
1. class Visitor {
2. public:
3.     virtual void visitRootNode(const RootNode * n) = 0;
4.     virtual void visitNode(const Node * n) = 0;
5.     virtual void visitFinalNode(const FinalNode * n) = 0;
6. };
7.
8. class TreePrintVisitor : public Visitor { ... };
9. class TreeToXMLExportVisitor : public Visitor { ... };
```

Add an accept function to the datastructure that takes a visitor.
→ Implement a visitor for each functionality you need.

```
1. class AbstractNode{
2.     int _data;
3. public:
4.     virtual void accept(Visitor* v) const = 0;
5. };
6.
7. class RootNode {
8.     vector<AbstractNode*> _children;
9. public:
10.    void accept(Visitor* v) const override {
11.        v->visitRootNode(this); };
12. };
13.
14. class Node : public AbstractNode {
15.     vector<AbstractNode*> _children;
16. public:
17.     void accept(Visitor* v) const override {
18.         v->visitNode(this); };
19. };
```

Visitor: Structure



Visitor: Benefits & Drawbacks

Benefits

- **Open/Closed Principle.** Introduce new behavior that can work with objects of different classes without changing these classes.
- **Single Responsibility Principle.** Move multiple versions of the same behavior into the same class.
- A visitor object can accumulate useful information while working with various objects.

Drawbacks

- Need to update all visitors each time a class gets added to the element hierarchy.
- Visitors might lack access to private fields.

Combining Patterns

Combine multiple patterns into more complex patterns.

Model-View-Controller



Combining the
Composite, Strategy, and Observer patterns

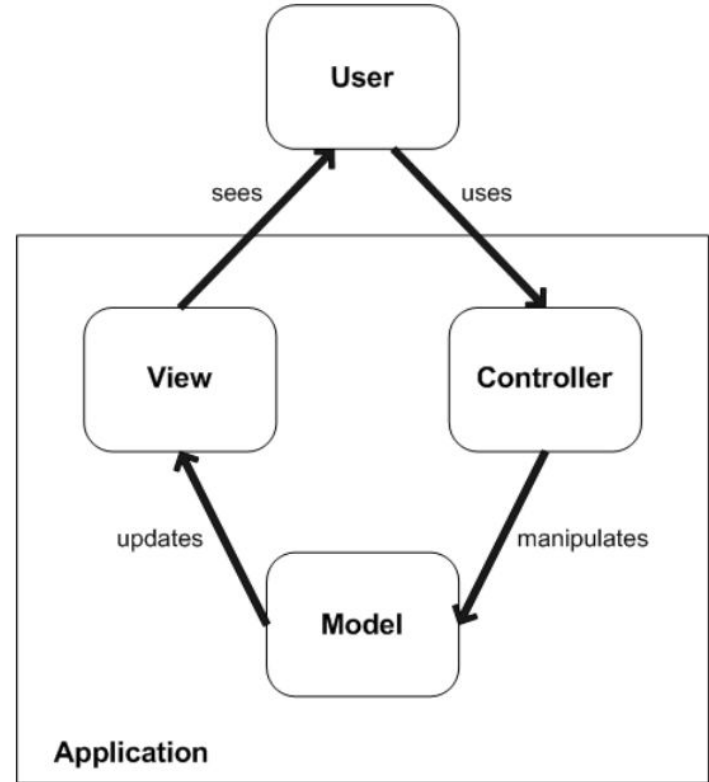
Model-View-Controller: Problem

- We want to develop a user interface for an application which allows the user to modify the data of the application.
- How can we implement this;
 - and keep the coupling between the graphical front-end and the database back-end low?
 - and satisfy the single responsibility principle?
Example: graphical widgets should not be responsible for both visualizing data and manipulating data.
 - and satisfy the open/closed principle?
Example: be able to adjust the UI without having to change the database.

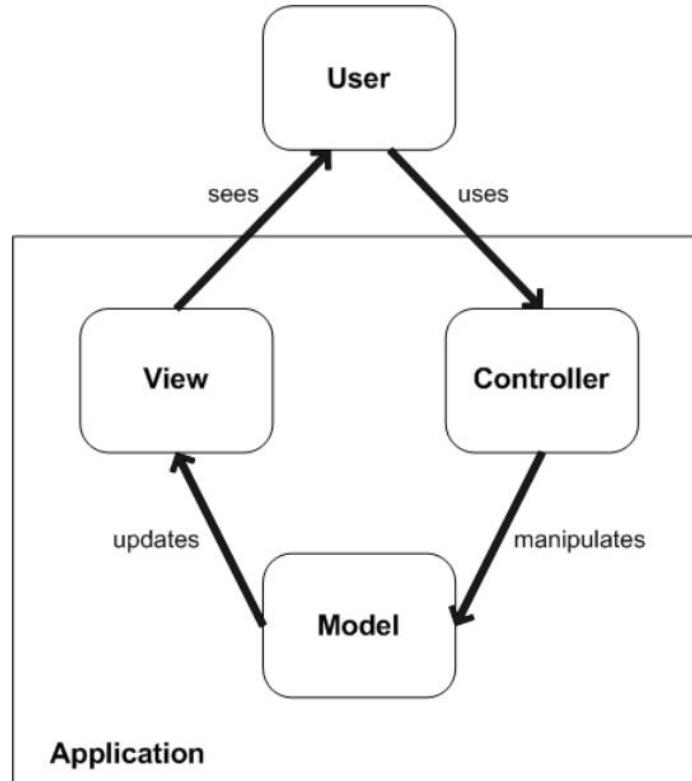
Model-View-Controller: Solution

Separate your application in three parts:

- The **Model** handles the back-end of your application. (database, simulation,...)
- The **View** handles the visual front-end of your application. (visualizing data, showing results,...)
- The **Controller** handles the user input of your application.



Model-View-Controller: Structure



MVC: Benefits & Drawbacks

Benefits

- **Open/Closed Principle.** Introduce new views or controllers without changing the model.
- **Single Responsibility Principle.** Separated the responsibility of showing data and handling user input.
- Low coupling between the front-end and the back-end.
- Can be implemented in a distributed system.

Drawbacks

- Increases the complexity and overhead in the application.

More patterns?

Free online resource: <https://refactoring.guru/design-patterns>

- describes more patterns

- detailed description of patterns, including examples